

This is a GROUP activity. You need to indicate the names of CONTRIBUTING members in the submission in the files themselves. Thus, for Part 1, include a text box at the upper left hand of the diagram that should contain the names of the contributors and the section. For Part 2 and Part 3, include the names of the contributors as authors in the Javadoc comment. Only 1 of the members need to upload. You may submit multiple times, but only the last submission (by the deadline) is going to be assessed.

Do the following items in sequence.

Part 1:

Draw the UML class diagram to model an online movie. Refer to details below.

Part 2:

Implement the class `Movie` based on the UML class diagram designed. Make sure that you include pertinent internal documentation to be useful for documentation generation via Javadoc.

Each online movie (let's say from iTunes) has a title, genre, year released, an array of at most 5 names of actors, and the number of times it was rented.

*We also want to keep track of the total number of movie objects. This is incremented every time a movie object is created.

Creating a movie instance can be via any of the following only: (1) title, genre, and year released are provided; (2) title and genre are provided; (3) title, genre, and 1 actor name are provided; and (4) title and year released are provided.

The title cannot be modified. The following are the only valid genres: drama, horror, comedy, biography, sci-fi, action, romance, and kids. If the given genre is invalid, the default assigned is comedy. The genre can still be updated later, if wanted. However, if the genre to be assigned is invalid, the last valid genre assigned to the same movie will be retained. If the year released is not provided, it is by default 2020. The number of times the movie was rented should initially be 0.

There must be a way to set or update the year too. There must be way to add an actor, but adding would fail if the array is already full. There is a `rent()` method for the movie that will increment the count for the number of times this movie is rented. There is also an `isBlockbuster()` method that will return true if the number of times the movie is rented is at least 10000. [For your testing purposes (i.e., prior to your submission), it is suggested that you use a smaller number.] There must be a way to retrieve the values of the data stored. Additionally, the `toString()` method is overridden to return the title, year, and genre. The `equals()` method is also overridden and it returns the result of comparing the current movie's and the parameter's title, genre, and year released.

*Lastly, create a static method that will decrement the total number of movie objects. This is usually called when a movie object is deleted.

Part 3:

Implement a driver called `MovieDriver` that will test your `Movie` class by performing the following in sequence:

1. Declare and instantiate an array of 5 `Movie` objects called `mustWatch`
2. Instantiate the first `Movie` object using the following information:
title = X-Men , genre = thriller, year = 2000
3. Instantiate the second `Movie` object using the following information:
title = Pokemon the movie , genre = kids
4. Instantiate the third `Movie` object using the following information:
title = Thor , genre = action , actor = Chris Hemsworth
5. Instantiate the fourth `Movie` object using the following information:
title = Frozen 2 , year = 2019
6. *Display the total number of movies instantiated
7. Display the title, genre, and year of all movies in `mustWatch`
8. Modify genre of first movie in `mustWatch` to sci-fi
9. Modify genre of 4th movie in `mustWatch` to kids
10. Modify year released of second movie in `mustWatch` to 2019
11. Modify year released of third movie in `mustWatch` to 2011
12. Display the title, genre, and year of all movies in `mustWatch`
13. Add Natalie Portman as another actor to the third movie stored in `mustWatch`
14. Add the following actors (in sequence) to the second movie stored in `mustWatch`:
Pikachu, Charizard, Squirtle, Ditto, Mewtwo, Psyduck
15. Display all actors of the third movie stored in `mustWatch`
16. Display all actors of the second movie stored in `mustWatch`
17. Declare and instantiate another array of at most 3 movies called `myRental`
18. Rent all movies that are in the genre of kids from `mustWatch`. These movies should now be associated also to `myRental`
19. *Display the total number of movies instantiated
20. Display the title, genre, and year of all movies in `mustWatch`
21. Display the title, genre, and year of all movies in `myRental`
22. Rent the movie Frozen 2 again and store that as the last element to `myRental`
23. Display the number of times the movie stored in the second element of `myRental` was rented and display the result of `isBlockBuster()` for that movie
24. Set `myRental` to null
25. Delete the movie object with title X-Men.
26. * Update the total number of movies instantiated. Display the total number of movies instantiated

Part 4: Generate the documentation of class `Movie` via Javadoc.